Reverse- and Reengineering
Homework 2

Winter 2026/27 — Prof. Dr. Christian Hammer
Date: 27.10.2026 — Dr. Mohammad Rezaalipour

This sheet is not for submission. However, we strongly encourage you to solve the problems and bring your solutions to the tutorials for discussion.

1 x86 Assembly

The two assembly programs below define two functions: foo and bar. Identify the function arguments and write the equivalent C programs.

Listing 1: `bar` function

```
1  push   rbp
2  mov    rbp, rsp
3  mov    dword ptr [rbp - 4], edi
4  mov    dword ptr [rbp - 8], esi
5  mov    eax, dword ptr [rbp - 4]
6  or     eax, dword ptr [rbp - 8]
7  xor    eax, -1
8  pop    rbp
9  ret
```

Listing 2: `foo` function

```
1  push   rbp
2  mov    rbp, rsp
3  mov    dword ptr [rbp - 4], edi
4  mov    dword ptr [rbp - 8], esi
5  mov    dword ptr [rbp - 12], edx
6  mov    eax, dword ptr [rbp - 4]
7  xor    eax, -1
8  mov    ecx, dword ptr [rbp - 8]
9  xor    ecx, -1
10 and    ecx, dword ptr [rbp - 12]
11 or     eax, ecx
12 pop    rbp
13 ret
```

2 Control Flow Graph (CFG)

The assembly program below defines two functions: `foo` and `main`.

1. Draw the intraprocedural CFG for the function `foo`.
2. What are the functions called from `main`?
3. Guess what the function `foo` computes.

```
foo(int):
    push   rbp
    mov    rbp, rsp
    mov    DWORD PTR [rbp-20], edi
    mov    DWORD PTR [rbp-4], 0
    mov    DWORD PTR [rbp-8], 1
    mov    DWORD PTR [rbp-12], 0
    cmp    DWORD PTR [rbp-20], 1
    jg     .L2
    mov    eax, DWORD PTR [rbp-4]
    mov    DWORD PTR [rbp-12], eax
    jmp    .L3
.L2:
    cmp    DWORD PTR [rbp-20], 2
    jne    .L4
    mov    eax, DWORD PTR [rbp-8]
    mov    DWORD PTR [rbp-12], eax
    jmp    .L3
.L4:
    mov    DWORD PTR [rbp-16], 3
    jmp    .L5
.L6:
    mov    edx, DWORD PTR [rbp-4]
    mov    eax, DWORD PTR [rbp-8]
    add    eax, edx
    mov    DWORD PTR [rbp-12], eax
    mov    eax, DWORD PTR [rbp-8]
    mov    DWORD PTR [rbp-4], eax
    mov    eax, DWORD PTR [rbp-12]
    mov    DWORD PTR [rbp-8], eax
    add    DWORD PTR [rbp-16], 1
.L5:
    mov    eax, DWORD PTR [rbp-16]
    cmp    eax, DWORD PTR [rbp-20]
    jle    .L6
.L3:
    mov    eax, DWORD PTR [rbp-12]
    pop    rbp
    ret
.LC0:
    .string "%d"
main:
    push   rbp
    mov    rbp, rsp
    sub    rsp, 16
    lea    rax, [rbp-8]
    mov    rsi, rax
    mov    edi, OFFSET FLAT:.LC0
    mov    eax, 0
    call   __isoc99_scanf
    mov    eax, DWORD PTR [rbp-8]
    mov    edi, eax
    call   foo(int)
    mov    DWORD PTR [rbp-4], eax
    mov    eax, DWORD PTR [rbp-4]
    mov    esi, eax
    mov    edi, OFFSET FLAT:.LC0
    mov    eax, 0
    call   printf
    mov    eax, 0
    leave
    ret
```